

pyAmpliCol 0.1.0

Methodology and Performance Report

Rikkert Frederix

Valentin Hirschi

Timea Vitos

Release 0.1.0

Measurement profile: `unscoped`

Abstract

This report presents the physics coverage, numerical validation, and performance of `pyAmpliCol` 0.1.0. We compare compiled, eager, and recurrence evaluation for the built-in Standard Model and for the same model imported from UFO/JSON, together with scalar-contact and scalar-gravity benchmarks. Every reported timing is associated with a documented software revision, hardware profile, numerical configuration, and a validation reference appropriate to the corresponding data set.

Contents

1	Scope	2
2	Current Recursion and Model Lowering	2
3	Colour and Runtime Semantics	3
4	Benchmark Methodology	3
5	Standard-Model Process Matrices	5
6	Z-plus-Jets Evaluator Ladders	35
7	Colour-Singlet Model Ladders	42
8	Interpretation	42
9	Worked $d\bar{d} \rightarrow Zgg$ Example	43
10	Shared-Current DAG and Numerical Execution	45
11	Reusable Leading-Colour Execution Layouts	47
12	Prepared-Model Eager DAG Execution	49
13	Supported UFO and Serialized-JSON Models	51

1 Scope

`pyAmpliCol` generates and evaluates tree-level scattering amplitudes. Compiled mode specializes a shared-current directed acyclic graph (DAG) to a process. Eager mode evaluates the same graph with reusable model kernels. Recurrence mode constructs a compact current schedule directly. The three modes implement the same physics normalization, colour treatment, helicity sums, and resolved observables. The original `AmpliCol` program and its event-generation setting are described in Ref. [5].

The report has three purposes:

1. define the observables and conditions required for a reproducible benchmark;
2. preserve a stable matrix of representative process families and colour approximations; and
3. present performance only after numerical agreement has been verified.

In a complete architecture edition, an `N/A` entry denotes a process/multiplicity combination outside the declared measurement scope or one to which the observable does not apply. Completion requires every applicable cell with at most four final-state legs to be numerical. A measurement that does not pass numerical validation is labelled explicitly and never replaced by a timing value. No value is copied, interpolated, or estimated from a neighbouring cell.

2 Current Recursion and Model Lowering

2.1 Shared-current DAG

For an external subset I , particle state t , and local spin or flow label χ , an off-shell current has the schematic recursion

$$J_{t,\chi}^\alpha(I) = P^\alpha_\beta(t, p_I) \sum_{I=A \dot{\cup} B} V^\beta_{\gamma\delta}(t_A, t_B \rightarrow t) J_{t_A, \chi_A}^\gamma(A) J_{t_B, \chi_B}^\delta(B), \quad p_I = \sum_{i \in I} p_i. \quad (1)$$

Current identity includes the particle, external labels, helicity ancestry, chirality or spin state, flavour and charge flow, colour state, momentum mask, and any auxiliary-current kind. Contributions with the same complete identity share one node. A topological sweep therefore evaluates each retained current once per phase-space point and reuses it in all dependent amplitudes. This is a Berends–Giele-type construction [1].

The Universal FeynRules Output (UFO) format describes particles, parameters, interactions, Lorentz tensors, colour tensors, and propagators in a model-independent form [3, 4]. `pyAmpliCol` normalizes a trusted UFO Python module, or its own serialized JSON representation of that model, to the same canonical representation used by process generation. The UFO module is executable Python; the corresponding serialized JSON form is data-only.

Optimizations on the external-model path are proof-gated rather than selected by particle names or PDG values. Exact normalized colour and Lorentz expressions, propagator metadata, and coupling relations certify current reflection, helicity, and reusable-kernel identities. For overlapping Standard-Model validation processes, the resulting current, interaction, amplitude-root, and colour-sector counts must match the built-in compatibility model after applying the same independently inferred perturbative-order envelope. An external compiler may retain fewer evaluator calls only when an additional exact kernel-equivalence certificate proves that reuse.

2.2 Contact terms and spin two

A supported vertex with valence above three is lowered to a deterministic tree of non-propagating auxiliary currents. The construction inserts the original coupling once and introduces no pole. Direct contraction of the original contact tensor is the validation reference for the scalar-contact ladder.

The scalar-gravity ladder exercises rank-two external states and momentum-dependent interactions. For a massless graviton, the helicity-two polarization tensor is represented as

$$\epsilon_{\pm 2}^{\mu\nu} = \epsilon_\pm^\mu \epsilon_\pm^\nu. \quad (2)$$

Validation checks symmetry, transversality, tracelessness, normalization, and agreement between ordinary and higher-precision evaluation.

3 Colour and Runtime Semantics

The process matrices use three colour contracts:

- LC** Leading-colour ordered amplitudes with physical colour-flow selectors [2].
- NLC** Every colour-metric entry whose leading power is at most two below the leading power of the complete basis.
- Full** The complete supported sparse colour metric for the generated amplitude basis.

For NLC, “retained” applies to a colour-matrix *entry*, not to a truncated numerical coefficient. Write the exact overlap of two colour-flow tensors as

$$C_{ab}(N_c) = \sum_p c_p^{(ab)} N_c^p, \quad p_{ab} = \max\{p : c_p^{(ab)} \neq 0\}, \quad (3)$$

and let P be the leading power of the complete colour basis. The sparse NLC metric is

$$C_{ab}^{\text{NLC}} = \begin{cases} C_{ab}(3), & p_{ab} \geq P - 2, \\ 0, & p_{ab} < P - 2. \end{cases} \quad (4)$$

Thus every qualifying entry keeps all of its finite- N_c terms. For three open fundamental lines and one adjoint, $P = 4$. Representative retained off-diagonal overlaps are

exact overlap	highest power	NLC at $N_c = 3$	full at $N_c = 3$
$-N_c^3 + N_c$	3	-24	-24
$N_c^2 - 1$	2	8	8

The powers and coefficients are retained exactly. When only the upper-triangular part of the colour metric is stored, off-diagonal interference terms receive the usual factor of two; this factor is not part of the colour coefficient in Eq. (4).

The summed observable returns one matrix element per point. Leading-colour resolved values have axes (**point**, **helicity**, **flow**); NLC and full-colour values have one contracted colour component per helicity. Summing all non-point axes must reproduce the directly evaluated total. Selectors refer to physical helicities and flows.

All public interfaces use the same numerical core. Cross-interface agreement therefore checks data conventions, while a separately generated value is required for an independent physics comparison.

The JIT backend stores a portable SymJIT application for binary64 evaluation; the receiving host creates code appropriate for its processor. ASM and C++ payloads are target-specific. Evaluation above binary64 precision uses the retained symbolic evaluator state.

4 Benchmark Methodology

4.1 Configuration contract

Each measurement begins from an immutable resolved configuration. Unless a table states otherwise, measurements use a batch size of 128, two warmup runs, at least five timed samples, a five-second sampling target, and one CPU core. Compiled process matrices use JIT O3; eager and recurrence matrices use portable JIT O2 prepared models. Explicitly labelled rows in the dedicated Z ladder use JIT O1, JIT O3, ASM O3, or C++ O3. Backend-specific optimization settings are retained with the measurement record.

Generation time begins with the resolved model input required by an execution mode and ends with a validated, loadable process representation. Compiled mode includes process-specific evaluator construction and backend compilation. Eager and recurrence modes include construction and serialization of their respective numerical schedules. Preparation of a reusable built-in or UFO/JSON model bundle occurs before this timer and is therefore excluded from the process-generation values in the tables.

Runtime measurements use the summed matrix element. The reported native wall time starts after caller-language array conversion and includes source construction, numerical evaluation, selector handling, and final reduction. The narrower execution submetric contains the mode-specific numerical schedule. Caller-language packing is excluded from both quantities. Comparisons use the same process, colour accuracy, parameters, phase-space point, precision, selectors, and normalization.

4.2 Benchmark platform

This source build is an unmeasured template; a published edition supplies architecture-specific environment metadata. Its hardware, operating system, and processor summary is platform metadata unavailable. The measurement profile is **unscoped**; its recorded software environment is toolchain metadata unavailable. Measurements from different processor architectures are published as separate editions and are never combined in a summary ratio. Unless a table states otherwise, generation and evaluation use one CPU core and a five-second sampling target.

4.3 Numerical validation

For each Standard-Model process, all compared execution modes and the original **AmpliCol** reference are evaluated at the same deterministic phase-space point with the same parameter card. Cross-mode comparisons use relative tolerance 10^{-12} and absolute tolerance 10^{-15} ; comparisons with the independent **AmpliCol** result use relative tolerance 10^{-8} . Scalar-model ladders instead compare binary64 evaluation with higher precision and, where applicable, direct contact-tensor contraction. Every result must reproduce its resolved sum and preserve expected structural zeros.

Timing and numerical validation are separate requirements: a timing is reported only when the corresponding matrix element passes all validation checks, and numerical agreement alone does not substitute for a measured generation or runtime value.

5 Standard-Model Process Matrices

The process matrices use recurrence as the primary `pyAmpliCol` execution mode. Built-in-SM and UFO-SM recurrence JIT O2 are compared with independent original-`AmpliCol` results. Separate built-in-SM matrices compare compiled JIT O3 and eager JIT O2 with recurrence JIT O2 for the same process, colour treatment, and observable.

The canonical matrix spans electroweak vector production, charged-current and leptonic channels, heavy-quark production, pure gluons, multi-boson final states, Higgs-associated final states, and three- and four-quark-line cases. Here n is the number of final-state particles. LC coverage retains the declared grid through $n = 9$, with the bounded families ending at the multiplicity stated by the process definition. NLC and full-colour grids retain $n = 1, \dots, 5$. Cells outside this architecture edition’s declared measurement scope or those to which a process definition does not apply are shown as `N/A`. A complete edition requires every applicable cell with $n \leq 4$ to be numerical. An in-scope result that does not pass numerical validation is identified by a status and does not carry a timing ratio.

The UFO-SM recurrence matrices use the same process definitions, colour contracts, numerical parameters, selectors, and validation points as the built-in-SM recurrence matrices. Their source is the packaged Standard Model loaded through the generic UFO/JSON compiler. This comparison tests model-generic lowering without changing the requested physics or the original `AmpliCol` reference. Compiled and eager process matrices are built-in-only; they measure execution-strategy differences against recurrence without introducing a second model-loading comparison.

Numerical validation summary

Only measurements whose numerical checks pass are counted below. The scope comprises every applicable report cell with at most four final-state particles.

final-state multiplicity	required	verified	coverage
1	64	0	incomplete (64 open)
2	186	0	incomplete (186 open)
3	206	0	incomplete (206 open)
4	286	0	incomplete (286 open)
total	742	0	incomplete (742 open)

validation comparison	passed	largest relative difference
independent original- <code>AmpliCol</code> reference records	0	reference
<code>pyAmpliCol</code> versus independent reference	0	—
compiled/eager versus recurrence	0	—
optimized result versus resolved sum	0	—
scalar result versus higher precision	0	—

Profile: `unscoped`. Measured source: `source identity is incomplete or mixed`. Independent-reference comparisons use a relative tolerance of 10^{-8} ; cross-mode, resolved-sum, and higher-precision comparisons use 10^{-12} .

Workloads and denominators

For a reported timing quantity T , each multiplier is

$$r_{\text{candidate/baseline}} = \frac{T_{\text{candidate}}}{T_{\text{baseline}}}. \quad (5)$$

Green, orange, and red denote respectively $r < 1$, $1 \leq r < 2$, and $r \geq 2$. Green therefore means that the candidate is faster than the baseline named in the table heading. The recurrence matrices use original **AmpliCol** as the baseline. The compiled and eager matrices use built-in-SM recurrence JIT O2 as the baseline.

Every LC comparison contains two complete-coverage layouts:

- topology replay, measured with one physical colour flow selected and all retained helicities summed; and
- the all-flow union, measured with one physical helicity selected and all retained colour flows summed.

Neither selection is fixed during generation. Each LC cell reports the two baseline generation quantities and wall times, followed by candidate generation entries and **(wall|execution)** runtime ratios. Compiled and eager generation are compared with recurrence using the same layout.

The two original-**AmpliCol** reference workloads have different setup boundaries. For the selected-flow workload, the reported generation quantity is creation of the process library. For the all-flow workload, it is preparation of the direct iMode-2 evaluation. Runtime ratios compare the same physical observable. The all-flow setup quantities are not like-for-like, so the table shows the candidate process-generation time as an absolute value marked **n.c.** (not comparable) instead of forming a ratio.

NLC and full colour use one helicity-summed, colour-contracted artifact per mode. Their cells report the absolute baseline generation and wall time, followed by the candidate generation ratio and **(wall|execution)** runtime ratios. No per-flow scalar is defined for these contracted colour accuracies.

The wall quantity is the repeated native evaluation time after caller-language input conversion. The execution quantity is the narrower numerical-kernel part of that measurement. Both are divided by the matching baseline wall time because the reference implementations do not expose a common internal submetric.

Preparing reusable model-level kernels is excluded from process-generation time and reported separately. Compiled generation still includes process-specific evaluator construction and backend compilation.

Publication requires recurrence to agree with original **AmpliCol** at the independent-reference tolerance, and compiled and eager results to agree with recurrence at the stricter cross-mode tolerance. Each result must also reproduce its resolved sum. LC comparisons use the same physical flow and helicity selections in both layouts. A failed validation has no timing ratio.

Cell and summary layout

contract	baseline entry	candidate entries
LC, upper	topology-replay/all-flow generation quantities in seconds	layout-matched ratios, except for the absolute n.c. all-flow candidate against original AmpliCol
LC, lower	selected-flow/all-flow wall times in microseconds per point	for each workload, (wall execution) ratios
NLC/full, upper	contracted generation time in seconds	candidate generation ratio
NLC/full, lower	contracted wall time in microseconds per point	(wall execution) ratios

Summary rows report the baseline mean over valid candidate–baseline pairs and the ratio of sums $\sum_i T_{\text{candidate},i} / \sum_i T_{\text{baseline},i}$. LC summaries remain separate for topology replay and all-flow union. Missing, unsupported, and validation-failed cells do not contribute to a ratio-of-sums summary.

The tables are grouped by execution mode, model source, and colour treatment.

5.1 Built-in SM recurrence versus AmpliCol LC

Built-in SM recurrence versus AmpliCol LC (block 1 of 3)

ID base process	n=1	n=2	n=3
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
summary: generation	N/A / N/A	N/A / N/A	N/A / N/A
summary: wall	N/A / N/A	N/A / N/A	N/A / N/A

Baseline: original AmpliCol; candidate: recurrence JIT O2. Each cell shows the selected-flow library-generation and all-flow direct-setup baseline quantities. The selected-flow entry carries a generation ratio; the all-flow entry carries the absolute pyAmpliCol process-generation time marked n.c. (not comparable). Both runtime entries retain their (wall|native execution) ratios.

Built-in SM recurrence versus AmpliCol LC (block 2 of 3)

ID base process	n=4	n=5	n=6
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
summary: generation	N/A / N/A	N/A / N/A	N/A / N/A
summary: wall	N/A / N/A	N/A / N/A	N/A / N/A

Baseline: original AmpliCol; candidate: recurrence JIT O2. Each cell shows the selected-flow library-generation and all-flow direct-setup baseline quantities. The selected-flow entry carries a generation ratio; the all-flow entry carries the absolute pyAmpliCol process-generation time marked n.c. (not comparable). Both runtime entries retain their (wall|native execution) ratios.

Built-in SM recurrence versus AmpliCol LC (block 3 of 3)

ID base process	n=7	n=8	n=9
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A
summary: generation	N/A / N/A	N/A / N/A	N/A / N/A
summary: wall	N/A / N/A	N/A / N/A	N/A / N/A

Baseline: original AmpliCol; candidate: recurrence JIT O2. Each cell shows the selected-flow library-generation and all-flow direct-setup baseline quantities. The selected-flow entry carries a generation ratio; the all-flow entry carries the absolute pyAmpliCol process-generation time marked n.c. (not comparable). Both runtime entries retain their (wall|native execution) ratios.

5.2 Built-in SM recurrence versus AmpliCol NLC

Built-in SM recurrence versus AmpliCol NLC (block 1 of 2)

ID base process	n=1	n=2	n=3
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
summary: generation	N/A	N/A	N/A
summary: wall	N/A	N/A	N/A

Baseline: original AmpliCol; candidate: recurrence JIT O2. Each cell shows the baseline generation and wall time, followed by candidate/baseline generation and (wall|native execution) ratios.

Built-in SM recurrence versus AmpliCol NLC (block 2 of 2)

ID base process	n=4	n=5
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A
summary: generation	N/A	N/A
summary: wall	N/A	N/A

Baseline: original AmpliCol; candidate: recurrence JIT O2. Each cell shows the baseline generation and wall time, followed by candidate/baseline generation and (wall|native execution) ratios.

5.3 Built-in SM recurrence versus AmpliCol full-colour

Built-in SM recurrence versus AmpliCol full-colour (block 1 of 2)

ID base process	n=1	n=2	n=3
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
summary: generation	N/A	N/A	N/A
summary: wall	N/A	N/A	N/A

Baseline: original AmpliCol; candidate: recurrence JIT O2. Each cell shows the baseline generation and wall time, followed by candidate/baseline generation and (wall|native execution) ratios.

Built-in SM recurrence versus AmpliCol full-colour (block 2 of 2)

ID base process	n=4	n=5
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A
summary: generation	N/A	N/A
summary: wall	N/A	N/A

Baseline: original AmpliCol; candidate: recurrence JIT O2. Each cell shows the baseline generation and wall time, followed by candidate/baseline generation and (wall|native execution) ratios.

5.4 UFO-SM recurrence versus AmpliCol LC

UFO-SM recurrence versus AmpliCol LC (block 1 of 3)

ID base process	n=1	n=2	n=3
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
summary: generation	N/A / N/A	N/A / N/A	N/A / N/A
summary: wall	N/A / N/A	N/A / N/A	N/A / N/A

Baseline: original AmpliCol; candidate: recurrence JIT O2. Each cell shows the selected-flow library-generation and all-flow direct-setup baseline quantities. The selected-flow entry carries a generation ratio; the all-flow entry carries the absolute pyAmpliCol process-generation time marked n.c. (not comparable). Both runtime entries retain their (wall|native execution) ratios.

UFO-SM recurrence versus AmpliCol LC (block 2 of 3)

ID base process	n=4	n=5	n=6
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
summary: generation	N/A / N/A	N/A / N/A	N/A / N/A
summary: wall	N/A / N/A	N/A / N/A	N/A / N/A

Baseline: original AmpliCol; candidate: recurrence JIT O2. Each cell shows the selected-flow library-generation and all-flow direct-setup baseline quantities. The selected-flow entry carries a generation ratio; the all-flow entry carries the absolute pyAmpliCol process-generation time marked n.c. (not comparable). Both runtime entries retain their (wall|native execution) ratios.

UFO-SM recurrence versus AmpliCol LC (block 3 of 3)

ID base process	n=7	n=8	n=9
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A
summary: generation	N/A / N/A	N/A / N/A	N/A / N/A
summary: wall	N/A / N/A	N/A / N/A	N/A / N/A

Baseline: original AmpliCol; candidate: recurrence JIT O2. Each cell shows the selected-flow library-generation and all-flow direct-setup baseline quantities. The selected-flow entry carries a generation ratio; the all-flow entry carries the absolute pyAmpliCol process-generation time marked n.c. (not comparable). Both runtime entries retain their (wall|native execution) ratios.

5.5 UFO-SM recurrence versus AmpliCol NLC

UFO-SM recurrence versus AmpliCol NLC (block 1 of 2)

ID base process	n=1	n=2	n=3
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
summary: generation	N/A	N/A	N/A
summary: wall	N/A	N/A	N/A

Baseline: original AmpliCol; candidate: recurrence JIT O2. Each cell shows the baseline generation and wall time, followed by candidate/baseline generation and (wall|native execution) ratios.

UFO-SM recurrence versus AmpliCol NLC (block 2 of 2)

ID base process	n=4	n=5
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A
summary: generation	N/A	N/A
summary: wall	N/A	N/A

Baseline: original AmpliCol; candidate: recurrence JIT O2. Each cell shows the baseline generation and wall time, followed by candidate/baseline generation and (wall|native execution) ratios.

5.6 UFO-SM recurrence versus AmpliCol full-colour

UFO-SM recurrence versus AmpliCol full-colour (block 1 of 2)

ID base process	n=1	n=2	n=3
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
summary: generation	N/A	N/A	N/A
summary: wall	N/A	N/A	N/A

Baseline: original AmpliCol; candidate: recurrence JIT O2. Each cell shows the baseline generation and wall time, followed by candidate/baseline generation and (wall|native execution) ratios.

UFO-SM recurrence versus AmpliCol full-colour (block 2 of 2)

ID base process	n=4	n=5
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A
summary: generation	N/A	N/A
summary: wall	N/A	N/A

Baseline: original AmpliCol; candidate: recurrence JIT O2. Each cell shows the baseline generation and wall time, followed by candidate/baseline generation and (wall|native execution) ratios.

5.7 Built-in SM compiled JIT O3 versus recurrence JIT O2 LC

Built-in SM compiled JIT O3 versus recurrence JIT O2 LC (block 1 of 3)

ID base process	n=1	n=2	n=3
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
summary: generation	N/A / N/A	N/A / N/A	N/A / N/A
summary: wall	N/A / N/A	N/A / N/A	N/A / N/A

Baseline: recurrence JIT O2; candidate: compiled JIT O3. Each cell shows the topology-replay/all-flow-union baseline generation times and wall times, followed by layout-matched candidate/baseline generation and (wall|native execution) ratios.

Built-in SM compiled JIT O3 versus recurrence JIT O2 LC (block 2 of 3)

ID base process	n=4	n=5	n=6
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
summary: generation	N/A / N/A	N/A / N/A	N/A / N/A
summary: wall	N/A / N/A	N/A / N/A	N/A / N/A

Baseline: recurrence JIT O2; candidate: compiled JIT O3. Each cell shows the topology-replay/all-flow-union baseline generation times and wall times, followed by layout-matched candidate/baseline generation and (wall|native execution) ratios.

Built-in SM compiled JIT O3 versus recurrence JIT O2 LC (block 3 of 3)

ID base process	n=7	n=8	n=9
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A
summary: generation	N/A / N/A	N/A / N/A	N/A / N/A
summary: wall	N/A / N/A	N/A / N/A	N/A / N/A

Baseline: recurrence JIT O2; candidate: compiled JIT O3. Each cell shows the topology-replay/all-flow-union baseline generation times and wall times, followed by layout-matched candidate/baseline generation and (wall|native execution) ratios.

5.8 Built-in SM compiled JIT O3 versus recurrence JIT O2 NLC

Built-in SM compiled JIT O3 versus recurrence JIT O2 NLC (block 1 of 2)

ID base process	n=1	n=2	n=3
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
summary: generation	N/A	N/A	N/A
summary: wall	N/A	N/A	N/A

Baseline: recurrence JIT O2; candidate: compiled JIT O3. Each cell shows the baseline generation and wall time, followed by candidate/baseline generation and (wall|native execution) ratios.

Built-in SM compiled JIT O3 versus recurrence JIT O2 NLC (block 2 of 2)

ID base process	n=4	n=5
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A
summary: generation	N/A	N/A
summary: wall	N/A	N/A

Baseline: recurrence JIT O2; candidate: compiled JIT O3. Each cell shows the baseline generation and wall time, followed by candidate/baseline generation and (wall|native execution) ratios.

5.9 Built-in SM compiled JIT O3 versus recurrence JIT O2 full-colour

Built-in SM compiled JIT O3 versus recurrence JIT O2 full-colour (block 1 of 2)

ID base process	n=1	n=2	n=3
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
summary: generation	N/A	N/A	N/A
summary: wall	N/A	N/A	N/A

Baseline: recurrence JIT O2; candidate: compiled JIT O3. Each cell shows the baseline generation and wall time, followed by candidate/baseline generation and (wall|native execution) ratios.

Built-in SM compiled JIT O3 versus recurrence JIT O2 full-colour (block 2 of 2)

ID base process	n=4	n=5
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A
summary: generation	N/A	N/A
summary: wall	N/A	N/A

Baseline: recurrence JIT O2; candidate: compiled JIT O3. Each cell shows the baseline generation and wall time, followed by candidate/baseline generation and (wall|native execution) ratios.

5.10 Built-in SM eager-DAG JIT O2 versus recurrence JIT O2 LC

Built-in SM eager-DAG JIT O2 versus recurrence JIT O2 LC (block 1 of 3)

ID base process	n=1	n=2	n=3
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
summary: generation	N/A / N/A	N/A / N/A	N/A / N/A
summary: wall	N/A / N/A	N/A / N/A	N/A / N/A

Baseline: recurrence JIT O2; candidate: eager-DAG JIT O2. Each cell shows the topology-replay/all-flow-union baseline generation times and wall times, followed by layout-matched candidate/baseline generation and (wall|native execution) ratios.

Built-in SM eager-DAG JIT O2 versus recurrence JIT O2 LC (block 2 of 3)

ID base process	n=4	n=5	n=6
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
summary: generation	N/A / N/A	N/A / N/A	N/A / N/A
summary: wall	N/A / N/A	N/A / N/A	N/A / N/A

Baseline: recurrence JIT O2; candidate: eager-DAG JIT O2. Each cell shows the topology-replay/all-flow-union baseline generation times and wall times, followed by layout-matched candidate/baseline generation and (wall|native execution) ratios.

Built-in SM eager-DAG JIT O2 versus recurrence JIT O2 LC (block 3 of 3)

ID base process	n=7	n=8	n=9
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A / N/A N/A N/A N/A / N/A N/A N/A	N/A
summary: generation	N/A / N/A	N/A / N/A	N/A / N/A
summary: wall	N/A / N/A	N/A / N/A	N/A / N/A

Baseline: recurrence JIT O2; candidate: eager-DAG JIT O2. Each cell shows the topology-replay/all-flow-union baseline generation times and wall times, followed by layout-matched candidate/baseline generation and (wall|native execution) ratios.

5.11 Built-in SM eager-DAG JIT O2 versus recurrence JIT O2 NLC

Built-in SM eager-DAG JIT O2 versus recurrence JIT O2 NLC (block 1 of 2)

ID base process	n=1	n=2	n=3
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
summary: generation	N/A	N/A	N/A
summary: wall	N/A	N/A	N/A

Baseline: recurrence JIT O2; candidate: eager-DAG JIT O2. Each cell shows the baseline generation and wall time, followed by candidate/baseline generation and (wall|native execution) ratios.

Built-in SM eager-DAG JIT O2 versus recurrence JIT O2 NLC (block 2 of 2)

ID	base process	n=4	n=5
1	$d\bar{d} \rightarrow Z + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
6	$gg \rightarrow t\bar{t} + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
8	$gg \rightarrow gg + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A
summary: generation		N/A	N/A
summary: wall		N/A	N/A

Baseline: recurrence JIT O2; candidate: eager-DAG JIT O2. Each cell shows the baseline generation and wall time, followed by candidate/baseline generation and (wall|native execution) ratios.

5.12 Built-in SM eager-DAG JIT O2 versus recurrence JIT O2 full-colour

Built-in SM eager-DAG JIT O2 versus recurrence JIT O2 full-colour (block 1 of 2)

ID base process	n=1	n=2	n=3
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
8 $gg \rightarrow gg + (n-2)g$	N/A	N/A N/A N/A N/A	N/A N/A N/A N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A N/A N/A N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
summary: generation	N/A	N/A	N/A
summary: wall	N/A	N/A	N/A

Baseline: recurrence JIT O2; candidate: eager-DAG JIT O2. Each cell shows the baseline generation and wall time, followed by candidate/baseline generation and (wall|native execution) ratios.

Built-in SM eager-DAG JIT O2 versus recurrence JIT O2 full-colour (block 2 of 2)

ID	base process	n=4	n=5
1	$d\bar{d} \rightarrow Z + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
6	$gg \rightarrow t\bar{t} + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
8	$gg \rightarrow gg + (n-2)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A N/A N/A N/A	N/A N/A N/A N/A
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A
summary: generation		N/A	N/A
summary: wall		N/A	N/A

Baseline: recurrence JIT O2; candidate: eager-DAG JIT O2. Each cell shows the baseline generation and wall time, followed by candidate/baseline generation and (wall|native execution) ratios.

6 Z-plus-Jets Evaluator Ladders

The dedicated $d\bar{d} \rightarrow Z + (n-1)g$ ladders compare the independent reference with compiled JIT O1, compiled JIT O3, ASM O3, C++ O3, eager-DAG JIT O2, and recurrence JIT O2 evaluator configurations. They retain final-state multiplicities $n = 1, \dots, 9$ for both the built-in model and the UFO-SM result set. Each row states its evaluator backend and optimization level explicitly; all other workload settings follow the benchmark contract above.

The eager and recurrence rows start from the corresponding portable JIT O2 prepared model. Each layout retains complete flow and helicity coverage; the selected-flow workload sums helicities, while the all-flow workload selects one helicity at runtime. Prepared-model creation is excluded from process-generation time. Every displayed Z -ladder ratio uses the matching original `AmpliCol` workload as its denominator, while the `pyAmpliCol` modes are also checked against one another at the stricter cross-mode tolerance.

Runtime comparisons between these backends also include their vectorization strategy. The `SymJIT` application evaluates a batch with SIMD across phase-space points, whereas generated ASM and C++ payloads are scalar and `Rusticol` invokes them point by point. Their ratios therefore measure both symbolic/code-generation quality and this SIMD advantage; they must not be interpreted as a comparison of otherwise identical vectorized kernels.

Worked $d\bar{d} \rightarrow Zg$ example

For $n = 2$, the explicit process is

$$d(p_1) \bar{d}(p_2) \rightarrow Z(p_3) g(p_4).$$

The report labels source particles in process order. The selected leading-colour reference order for the selected-flow timing is therefore $(2, 4, 1, 3)$: the colour line enters through $\bar{d}$, traverses the gluon, and exits through d , while the colour-singlet Z remains in the public ordering only to keep source labels unambiguous. The corresponding coloured word is $(2, 4, 1)$.

`pyAmpliCol` builds a shared-current representation for the selected process. Source nodes carry particle, momentum, helicity, flavour, and colour-flow labels; composite nodes are identified by their complete quantum state. The selected-flow workload uses the traversal $(2, 4, 1)$ and sums helicities; the all-flow workload sums every LC flow for one selected helicity. Both layouts retain complete physical coverage and are validated against the same reference point before timing.

6.1 Built-in SM dedicated $d\bar{d} \rightarrow Z + \text{gluon}$ performance

Built-in SM $d\bar{d} \rightarrow Z + \text{gluon}$ ladder (block 1 of 3)

n setup	selected flow, helicity sum			all flows, single helicity		
	gen [s]	wall [us/pt]	exec [us/pt]	gen [s]	wall [us/pt]	exec [us/pt]
1 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
1 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
1 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
1 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
1 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
1 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
1 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
2 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
2 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
2 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
2 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
2 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
2 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
2 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
3 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
3 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
3 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
3 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
3 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
3 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
3 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A

Original AmpliCol is the denominator. Each pyAmpliCol row reports separate topology-replay and all-flow-union generation and runtime measurements. Parenthesized values are candidate/reference ratios. All-flow generation shows the absolute pyAmpliCol value marked n.c. because its setup boundary differs from the reference.

Built-in SM $d\bar{d} \rightarrow Z + \text{gluon ladder}$ (block 2 of 3)

n setup	selected flow, helicity sum			all flows, single helicity		
	gen [s]	wall [us/pt]	exec [us/pt]	gen [s]	wall [us/pt]	exec [us/pt]
4 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
4 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
4 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
4 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
4 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
4 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
4 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
5 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
5 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
5 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
5 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
5 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
5 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
5 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
6 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
6 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
6 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
6 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
6 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
6 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
6 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A

Original AmpliCol is the denominator. Each pyAmpliCol row reports separate topology-replay and all-flow-union generation and runtime measurements. Parenthesized values are candidate/reference ratios. All-flow generation shows the absolute pyAmpliCol value marked n.c. because its setup boundary differs from the reference.

Built-in SM $d\bar{d} \rightarrow Z + \text{gluon ladder}$ (block 3 of 3)

n setup	selected flow, helicity sum			all flows, single helicity		
	gen [s]	wall [us/pt]	exec [us/pt]	gen [s]	wall [us/pt]	exec [us/pt]
7 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
7 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
7 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
7 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
7 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
7 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
7 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
8 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
8 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
8 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
8 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
8 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
8 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
8 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
9 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
9 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
9 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
9 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
9 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
9 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
9 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A

Original AmpliCol is the denominator. Each pyAmpliCol row reports separate topology-replay and all-flow-union generation and runtime measurements. Parenthesized values are candidate/reference ratios. All-flow generation shows the absolute pyAmpliCol value marked n.c. because its setup boundary differs from the reference.

6.2 UFO-SM dedicated $d\bar{d} \rightarrow Z + \text{gluon}$ performance

UFO-SM $d\bar{d} \rightarrow Z + \text{gluon}$ ladder (block 1 of 3)

n setup	selected flow, helicity sum			all flows, single helicity		
	gen [s]	wall [us/pt]	exec [us/pt]	gen [s]	wall [us/pt]	exec [us/pt]
1 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
1 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
1 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
1 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
1 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
1 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
1 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
2 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
2 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
2 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
2 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
2 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
2 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
2 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
3 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
3 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
3 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
3 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
3 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
3 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
3 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A

Original AmpliCol is the denominator. Each pyAmpliCol row reports separate topology-replay and all-flow-union generation and runtime measurements. Parenthesized values are candidate/reference ratios. All-flow generation shows the absolute pyAmpliCol value marked n.c. because its setup boundary differs from the reference.

UFO-SM $d\bar{d} \rightarrow Z + \text{gluon ladder}$ (block 2 of 3)

n setup	selected flow, helicity sum			all flows, single helicity		
	gen [s]	wall [us/pt]	exec [us/pt]	gen [s]	wall [us/pt]	exec [us/pt]
4 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
4 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
4 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
4 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
4 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
4 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
4 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
5 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
5 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
5 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
5 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
5 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
5 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
5 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
6 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
6 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
6 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
6 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
6 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
6 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
6 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A

Original AmpliCol is the denominator. Each pyAmpliCol row reports separate topology-replay and all-flow-union generation and runtime measurements. Parenthesized values are candidate/reference ratios. All-flow generation shows the absolute pyAmpliCol value marked n.c. because its setup boundary differs from the reference.

UFO-SM $d\bar{d} \rightarrow Z + \text{gluon ladder}$ (block 3 of 3)

n setup	selected flow, helicity sum			all flows, single helicity		
	gen [s]	wall [us/pt]	exec [us/pt]	gen [s]	wall [us/pt]	exec [us/pt]
7 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
7 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
7 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
7 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
7 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
7 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
7 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
8 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
8 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
8 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
8 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
8 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
8 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
8 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
9 AmpliCol reference	N/A	N/A	N/A	N/A	N/A	N/A
9 compiled JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
9 compiled ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
9 compiled C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
9 compiled JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
9 eager-DAG JIT O2	N/A	N/A	N/A	N/A	N/A	N/A
9 recurrence JIT O2	N/A	N/A	N/A	N/A	N/A	N/A

Original AmpliCol is the denominator. Each pyAmpliCol row reports separate topology-replay and all-flow-union generation and runtime measurements. Parenthesized values are candidate/reference ratios. All-flow generation shows the absolute pyAmpliCol value marked n.c. because its setup boundary differs from the reference.

Each LC workload reports process-generation time in seconds and native wall time and execution time in microseconds per phase-space point. Compiled rows use the labelled JIT, ASM, or C++ backend; eager and recurrence rows use portable JIT O2 prepared models. Parenthesized runtime values and selected-flow generation values compare each pyAmpliCol row with the corresponding AmpliCol reference workload. All-flow generation shows the absolute pyAmpliCol value marked n.c. because its setup boundary differs from the reference.

7 Colour-Singlet Model Ladders

Both colour-singlet ladders use compiled JIT O3 evaluation and the full-colour contract. The scalar-contact ladder retains $n = 2, \dots, 8$ final scalars and tests generic high-valence contact decomposition, identical-particle normalization, and source parity without Standard-Model-specific kernels. The scalar-gravity ladder retains $n = 2, \dots, 4$ final gravitons and exercises spin-two wavefunctions, tensor propagators, and momentum-dependent vertices. Because every external particle in these fixtures is a colour singlet, LC, NLC, and full colour reduce to the same unit colour contraction.

For these ladders, the relative-difference row is $||\mathcal{M}|_{\text{f64}}^2 - |\mathcal{M}|_{\text{hp}}^2| / \max(|\mathcal{M}|_{\text{hp}}^2, 10^{-300})$, where the high-precision value uses the same phase-space point. It is a numerical stability check, not a performance ratio.

7.1 Scalar-contact ladder

metric	scalar_0 scalar_0 > n*scalar_0						
	n=2	n=3	n=4	n=5	n=6	n=7	n=8
generation [s]	N/A	N/A	N/A	N/A	N/A	N/A	N/A
wall [$\mu\text{s}/\text{pt}$]	N/A	N/A	N/A	N/A	N/A	N/A	N/A
execution [$\mu\text{s}/\text{pt}$]	N/A	N/A	N/A	N/A	N/A	N/A	N/A
matrix element	N/A	N/A	N/A	N/A	N/A	N/A	N/A
relative diff. vs hp	N/A	N/A	N/A	N/A	N/A	N/A	N/A

7.2 Scalar-gravity ladder

metric	scalar_0 scalar_0 > n*graviton		
	n=2	n=3	n=4
generation [s]	N/A	N/A	N/A
wall [$\mu\text{s}/\text{pt}$]	N/A	N/A	N/A
execution [$\mu\text{s}/\text{pt}$]	N/A	N/A	N/A
matrix element	N/A	N/A	N/A
relative diff. vs hp	N/A	N/A	N/A

8 Interpretation

Performance comparisons are meaningful only for a baseline measured with the same observables, parameters, selectors, and hardware profile. The result tables therefore present validated, architecture-local comparisons; they do not combine measurements from different machines. An N/A entry is outside this edition’s measured scope or not applicable and must not be interpreted as zero. Numerical agreement, timing uncertainty, and source identity remain part of the published result set even when a compact table shows only the central timing.

9 Worked $d\bar{d} \rightarrow Zgg$ Example

This example illustrates how a familiar tree-level process is represented by reusable off-shell currents. It also fixes the momentum, helicity, and colour conventions used in the performance tables.

9.1 External legs and conventions

The physical process is

$$d(p_1) \bar{d}(p_2) \longrightarrow Z(p_3) g(p_4) g(p_5), \quad p_1 + p_2 = p_3 + p_4 + p_5. \quad (6)$$

The current recursion uses the all-outgoing convention

$$q_1 = -p_1, \quad q_2 = -p_2, \quad q_i = p_i \quad (i = 3, 4, 5), \quad \sum_{i=1}^5 q_i = 0. \quad (7)$$

External momenta are nevertheless supplied in the physical convention of Eq. (6); crossing is applied internally.

label	physical leg	outgoing state	momentum	states	colour role
1	incoming d	outgoing $\bar{d}$	$q_1 = -p_1$	2	$\bar{\mathbf{3}}$, line end
2	incoming $\bar{d}$	outgoing d	$q_2 = -p_2$	2	$\mathbf{3}$, line start
3	outgoing Z	outgoing Z	$q_3 = p_3$	3	singlet
4	outgoing g	outgoing g	$q_4 = p_4$	2	adjoint insertion
5	outgoing g	outgoing g	$q_5 = p_5$	2	adjoint insertion

There are $2 \times 2 \times 3 \times 2 \times 2 = 48$ physical helicity configurations. Twenty-four vanish identically for the chiral source and kernel conventions; they remain explicit zero entries in resolved results.

At leading colour the Z is absent from the colour word. Complete coverage contains the two orderings

$$c_0 = (2, 4, 5, 1), \quad c_1 = (2, 5, 4, 1). \quad (8)$$

Both are retained during generation, so either flow can be selected at evaluation time.

9.2 Reusable current recursion

For a nonempty source subset I , define

$$q_I = \sum_{i \in I} q_i, \quad m(I) = \sum_{i \in I} 2^{i-1}. \quad (9)$$

A one-leg current is the appropriate external spinor, polarization vector, or scalar wavefunction. Composite currents obey the binary recursion

$$J_{t,\Omega}^a(I) = P_t^a{}_b(q_I) \sum_{\substack{I=A \dot{\cup} B \\ A, B \neq \emptyset}} \sum_{v: t_A t_B \rightarrow t} \kappa_v V_v^b{}_{cd}(q_A, q_B) J_{t_A, \Omega_A}^c(A) J_{t_B, \Omega_B}^d(B). \quad (10)$$

Here t is the particle state, P_t is its propagator, V_v is an allowed interaction, and Ω collects the quantum numbers required to identify the current. Only combinations consistent with the model, perturbative order, and colour ordering enter the sum.

Representative terms along the quark line are

$$J_d(2, 5) = P_d V_{dgd} [J_d(2), J_g(5)], \quad (11)$$

$$J_g(4, 5) = P_g V_{ggg} [J_g(4), J_g(5)], \quad (12)$$

$$J_d(2, 4, 5) = P_d \left(V_{dgd} [J_d(2, 5), J_g(4)] + V_{dgd} [J_d(2, 4), J_g(5)] + V_{dgd} [J_d(2), J_g(4, 5)] \right), \quad (13)$$

$$J_d(2, 3, 4, 5) = P_d \sum_{A \dot{\cup} B = \{2, 3, 4, 5\}} V [J(A), J(B)]. \quad (14)$$

Equation (14) includes every admissible placement of the Z emission and both gluon insertions. Ordered currents such as $J_d(2, 5, 4)$ and $J_d(2, 4, 5)$ remain distinct even though their momentum subset is the same.

The final closure joins the endpoint current to source label 1:

$$\mathcal{A}_{h,c}^{(r)} = C_{aa}^{(r)} J_d^a(1) J_d^a(2, 3, 4, 5), \quad \mathcal{A}_{h,c} = \sum_{r \in G(h,c)} \mathcal{A}_{h,c}^{(r)}. \quad (15)$$

The set $G(h, c)$ contains contributions that must be summed coherently. Identical subcurrents are evaluated once and reused wherever their complete quantum-number and ordering labels agree. This sharing is the essential difference between the current DAG and an independent evaluation of every Feynman diagram.

9.3 Resolved observable and normalization

The two leading-colour layouts described in Sec. 11 evaluate the same complete set of helicity and flow components. One is optimized for a selected flow with summed helicities; the other shares work across all flows for a selected helicity. Their resolved components and their summed matrix element are numerically identical.

For the built-in compatibility model, the normalization used in this example is

$$|\mathcal{M}|^2 = \mathcal{N} \sum_{h,c} w_{h,c} \left| \sum_{r \in G(h,c)} \mathcal{A}_{h,c}^{(r)} \right|^2, \quad \mathcal{N} = \frac{27 (4\pi\alpha_s)^2 (8\pi\alpha_{EW})}{36 \times 2}. \quad (16)$$

The denominator averages over the incoming colour and spin states, while the factor $2 = 2!$ accounts for the identical final-state gluons. Coupling powers, colour weights, coherent groups, and symmetry factors are retained separately so that changing numerical model parameters does not alter the process definition.

10 Shared-Current DAG and Numerical Execution

The current recursion becomes a directed acyclic graph (DAG) by merging physically identical intermediate currents. This section describes the identity, staging, and timing conventions without depending on a particular artifact format or software interface.

10.1 Current identity

Two contributions share a current only when every physics-relevant entry in

$$\Omega(J) = (t, I, H, \chi, s, F, Q, C, O, a, \pi_I) \quad (17)$$

is equal. These entries specify the particle species t , external subset I , helicity ancestry H , chirality and spin state (χ, s) , flavour and additive quantum-number flow (F, Q) , colour state C , perturbative order O , any auxiliary-current type a , and the colour-sensitive ordering π_I . The subset fixes the current momentum $q_I = \sum_{i \in I} q_i$; the ordered labels prevent distinct colour traversals from being merged.

Exact equality permits reuse across all amplitude contributions and physical flows. A second level of reuse applies when two target currents require the same kernel value up to a known sign or scalar factor. It is useful to keep the three concepts distinct:

$$\underbrace{\text{same current identity,}}_{\text{one stored node}} \quad \underbrace{\text{same kernel evaluation,}}_{\text{one numerical value}} \quad \underbrace{\text{same physical flow.}}_{\text{one resolved component}} \quad (18)$$

10.2 Topological stages and storage

The graph is evaluated in increasing source-subset size. An interaction forming a current from k external labels belongs to stage k , so every input is available before its consumer. For the process of Sec. 9, the stage sequence is schematically

$$\boxed{\text{sources}} \longrightarrow \boxed{|I| = 2} \longrightarrow \boxed{|I| = 3} \longrightarrow \boxed{|I| = 4} \longrightarrow \boxed{\text{amplitude roots}}. \quad (19)$$

At each stage, every admissible partition contributing to the same current is summed coherently before the current is propagated. The amplitude roots then close the largest retained currents against the remaining external states. This ordering makes every dependency explicit without assigning significance to storage locations or evaluator-specific identifiers.

A current may be needed as a source value, before propagation, or after propagation. Keeping these storage variants explicit prevents a propagator from being applied twice and lets the final closure request the correct form. Conceptually, the numerical state is

$$x = [x_{\text{current}} \mid x_{\text{momentum}} \mid x_{\text{model}}]. \quad (20)$$

Each stage consumes only the entries it needs:

$$x_k = L_k x, \quad y_k = E_k(x_k), \quad x[S_k] \leftarrow y_k, \quad (21)$$

where L_k gathers the stage-local inputs and S_k identifies its output slots. This stage-local layout reduces data movement while leaving the physics graph unchanged.

10.3 Compiled and eager evaluation

In compiled mode, a stage is optimized as a multi-output expression. For target current j ,

$$y_j = \mathcal{P}_j \left[\sum_{e: \text{target}(e)=j} f_e K_{g(e)}(x_k) \right], \quad (22)$$

where $K_{g(e)}$ is a reusable kernel value, f_e is the exact attachment factor, and $\mathcal{P}_j$ is the propagator or identity. Symbolic optimization may therefore share algebra across several current components within the stage.

Eager mode evaluates the same graph with prepared model kernels and explicit gather, accumulation, finalization, and closure steps. Recurrence mode builds a compact current schedule directly. The three modes preserve the same stage dependencies, coherent root sums, colour contraction, symmetry weights, and normalization; only the placement of symbolic compilation and numerical scheduling differs.

For leading-colour resolved evaluation, the public tensor is

$$R_{n,h,c} = |\mathcal{M}_{h,c}(p_n)|^2, \quad \text{shape}(R) = (N_{\text{point}}, N_{\text{helicity}}, N_{\text{flow}}). \quad (23)$$

Structural zeros remain explicit. Summing the resolved tensor reproduces the directly evaluated total within the validation tolerance.

10.4 Timing boundary

The report separates the complete native evaluation from its numerical-kernel submetric. The complete time begins after caller-language array conversion and includes source construction, momentum and parameter preparation, stage input gathers, kernel execution, output assignment, closures, and final reduction. The narrower execution time contains the mode-specific numerical kernels:

$$t_{\text{exec}} = \sum_k t(E_k) + t(E_{\text{root}}), \quad t_{\text{wall}} = t_{\text{exec}} + t_{\text{source}} + t_{\text{momentum/model}} + t_{\text{gather/scatter}} + t_{\text{reduce}} + t_{\text{batch}}. \quad (24)$$

This distinction exposes orchestration overhead while keeping the complete wall-time observable comparable across execution modes.

11 Reusable Leading-Colour Execution Layouts

Leading-colour coverage and its numerical layout are separate concepts. Let $\mathcal{H}$ be the retained helicities and $\mathcal{F}$ the retained physical colour flows. Complete leading-colour coverage represents

$$\mathcal{C}_{\text{LC}} = \mathcal{H} \times \mathcal{F} \quad (25)$$

and can expose every resolved amplitude $\mathcal{A}_{hf}$. For selected subsets $S_H \subseteq \mathcal{H}$ and $S_F \subseteq \mathcal{F}$, the observable is

$$\mathcal{W}(S_H, S_F) = \mathcal{N} \sum_{h \in S_H} \sum_{f \in S_F} w_{hf} \left| \sum_{r \in G(h, f)} \mathcal{A}_{hf}^{(r)} \right|^2. \quad (26)$$

Here $G(h, f)$ is a coherent contribution group, w_{hf} contains exact symmetry and leading-colour weights, and $\mathcal{N}$ is the process normalization. The layout changes how this sum is scheduled, not its components or normalization.

11.1 Two complete-coverage layouts

The report measures two layouts because selected-flow and all-flow workloads offer different opportunities for sharing:

property	topology replay	all-flow union
Measured workload	one selected flow, summed helicities	all flows, one selected helicity
Retained physics	every flow and helicity	every flow and helicity
Main reuse	equivalent flow topologies related by exact permutations and factors	currents and closures shared directly across all flows
Availability	compiled, eager, and recurrence	compiled, eager, and recurrence

In topology replay, exact relations partition physical flows into equivalence classes. If $\rho(f)$ represents the class containing f , the recurrence records a permutation π_f , an exact factor s_f , and the corresponding weight:

$$\mathcal{A}_{hf}(p) = s_f \mathcal{A}_{\pi_f(h), \rho(f)}(\pi_f(p)). \quad (27)$$

This avoids constructing an independent recurrence for every physical ordering and is suited to a selected-flow, helicity-summed evaluation.

The all-flow union instead merges the dependency graphs of all resolved components:

$$D_{\text{union}} = \left(\bigcup_{h \in \mathcal{H}, f \in \mathcal{F}} D_{hf} \right) / \sim, \quad (28)$$

where $\sim$ identifies currents, kernel values, and closures that are algebraically equal up to retained exact factors. For a selected helicity, one graph traversal then reuses intermediate currents across all flows.

Compiled mode specializes the chosen layout to process-wide stage evaluators. Eager mode applies prepared model kernels to the same dependency structure, and recurrence mode constructs a compact current schedule directly. These execution choices do not change $\mathcal{H}$, $\mathcal{F}$, or Eq. (26).

11.2 Runtime selection

Flow and helicity selections refer to physical components. A selection may be applied to a complete batch or specified independently for each point; omitting an axis sums all retained entries on that axis. Rows with equal selections can be grouped into contiguous numerical batches. This grouping changes neither the generated coverage nor the resolved result.

11.3 Generation and reference boundaries

The two layouts build different graphs and numerical schedules, so their process-generation times are distinct observables:

$$T_{\text{gen}}^{\text{replay}} \quad \text{and} \quad T_{\text{gen}}^{\text{union}}. \quad (29)$$

Cross-mode comparisons use the same layout on both sides. Preparation of reusable model-level kernels occurs before the process-generation timer and is not included in these table values.

The original-**AmpliCol** reference uses two different published routes. For the selected-flow workload, its generation value measures creation of the table-driven process library. For the all-flow workload, the reported setup value measures preparation of direct iMode-2 evaluation. The corresponding runtime measurements implement the same physical workloads as **pyAmpliCol**, but the all-flow setup quantities do not define a like-for-like process-generation speedup. The corresponding table therefore reports both quantities absolutely and marks the **pyAmpliCol** entry as not comparable.

11.4 NLC and full colour

The alternative layouts apply only at leading colour. NLC and full-colour evaluation construct an amplitude basis and contract it with the appropriate colour metric:

$$\mathcal{W}_{\text{contracted}} = \mathcal{N} \sum_h \sum_{a,b} \mathcal{A}_{ha}^* C_{ab} \mathcal{A}_{hb}. \quad (30)$$

These treatments do not expose the physical leading-colour flow axis, so there is no topology-replay or all-flow-union distinction.

12 Prepared-Model Eager DAG Execution

This section compares the compiled and eager realizations of the shared-current DAG; recurrence execution is described by the general recursion in Sec. 9. Compiled mode combines many current relations into process-specific stage evaluators. Eager mode prepares model-local kernels once and represents each process by an invocation schedule. They use the same physical graph, normalization, colour contraction, and resolved observables.

	Compiled mode	Eager mode
Numerical unit	Multi-output expression specialized to one process stage	Reusable model kernel applied by a process schedule
Process preparation	Builds and compiles process-specific stage evaluators	Maps the proven DAG to reusable model kernels and a process schedule
Runtime work	Calls a small number of wide stage evaluators	Applies reusable kernels and combines the resulting currents
Expected trade-off	More preparation for greater process-wide fusion	Faster process preparation with additional scheduling work

12.1 Prepared model and process schedule

A prepared model separates model-level algebra from process-level graph construction. Schematically,

$$\mathcal{B}_{M,b,o,t} = \left(\mathcal{I}_M, \{K_{\kappa}^{(b,o,t)}\}_{\kappa \in \mathcal{K}_M}, \mathcal{P}_M \right), \quad (31)$$

where $\mathcal{I}_M$ defines the model, K_{κ} are numerical kernels for its canonical interaction classes, and $\mathcal{P}_M$ records the numerical and target conventions. The labels b, o, t identify the backend, optimization, and target architecture.

For a process P , the eager schedule records the graph dependencies, the kernel associated with each interaction, the current transformations, and the final colour and helicity reductions. It also retains the dependencies needed for resolved helicity and colour-flow selections.

Because the numerical kernels are already part of $\mathcal{B}$, eager process generation does not compile a new evaluator for every process. Model preparation occurs before the process-generation timer and is not included in the process-generation values reported here.

12.2 Stage execution

Let $V_{c\alpha p}$ denote component α of current c at phase-space point p . For one stage, unpropagated components are

$$U_{c\alpha p}^{(s)} = \sum_{r \in \mathcal{R}_s(c, \alpha)} w_r \left[K_{\kappa(r)} \left(G_r(V_{\cdot p}), q_r(p), \theta \right) \right]_{\alpha(r)}, \quad (32)$$

where G_r gathers the required current components and momenta, w_r is an exact factor, and θ denotes model parameters. Contributions are accumulated before finalization:

$$V_{c\beta p}^{(s)} = \left[F_{\phi(c)} \left(U_{c \cdot p}^{(s)}, Q_c(p), \theta \right) \right]_{\beta}. \quad (33)$$

The map $F_{\phi(c)}$ is an identity, propagator, or another model-defined current transformation. Applying it once per current preserves the normalization convention of compiled mode.

After the last stage, closure relations produce amplitude components,

$$A_{h\gamma p} = \sum_{r \in \mathcal{C}(h, \gamma)} w_r^{\text{cl}} K_{\kappa(r)}^{\text{cl}} \left(G_r^{\text{cl}}(V_{\cdot p}), q_r(p), \theta \right), \quad (34)$$

and the standard colour/helicity reduction gives the requested observable. In a contracted colour basis,

$$\mathcal{M}_p = \mathcal{N} \sum_h \sum_{\gamma, \delta} A_{h\gamma p} C_{\gamma\delta} A_{h\delta p}^*, \quad (35)$$

with the same colour matrix, symmetry factors, initial-state average, and coupling normalization as the other execution modes.

12.3 Batched evaluation and selectors

Compatible kernel applications are batched across phase-space points so that the point dimension can be vectorized. Large input batches may be partitioned, but this changes neither the graph nor the numerical result.

A helicity or flow selection retains the complete dependency closure of the requested amplitudes. A complete leading-colour process therefore supports a selected flow with summed helicities, a selected helicity with all flows, or any retained subset without changing its physics content. The two corresponding graph layouts are described in Sec. 11.

12.4 Precision, portability, and interpretation

A standalone eager process contains the numerical kernels and schedule needed for evaluation. Portable JIT state is compiled for the receiving processor at load time; target-native ASM and C++ payloads remain architecture-specific. The exact symbolic representation is retained for higher-precision evaluation.

Eager generation time starts from a prepared model and ends with a validated, loadable process; prepared-model creation is outside this timing boundary. At runtime, eager mode avoids process-specific backend compilation but performs explicit scheduling and reusable-kernel dispatch. Compiled mode can be faster when process-wide fusion dominates, whereas eager mode can reduce preparation cost when many processes share one model. The tables quantify this trade-off under identical physics and validation conditions rather than assuming a universal ranking.

13 Supported UFO and Serialized-JSON Models

The external-model path compiles a supported subset of the UFO model format rather than interpreting arbitrary model code during amplitude evaluation. A trusted UFO Python module or its serialized JSON representation is normalized to a typed, model-independent representation before current recursion begins. An interaction is executable only when its tensors can be materialized and every transformation required for its evaluation can be checked algebraically.

13.1 Common loading and normalization route

UFO and JSON inputs enter the same compilation route,

$$\{\text{trusted UFO module, serialized JSON}\} \longrightarrow \text{normalized model} \longrightarrow \text{executable kernels.} \quad (36)$$

The UFO form is executable Python and must be obtained from a trusted source. The JSON form is data-only and is the preferred portable representation for exchange and archival. After loading, both forms use the same particle, parameter, propagator, tensor, and coupling-order definitions.

Model symbols occupy a model-local namespace, allowing two models with equally named parameters to coexist without identifying those parameters. The normalized representation contains typed particles, parameters, propagators, vertex terms, oriented kernels, component orderings, contractions, and Goldstone relations. Unsupported spin, colour, or function structures are rejected before process generation.

13.2 From a UFO vertex to current kernels

A UFO vertex is a matrix over colour and Lorentz structures:

$$\mathcal{V}(1, \dots, n) = \sum_{a,b} g_{ab} C_a(c_1, \dots, c_n) L_b(s_1, p_1; \dots; s_n, p_n). \quad (37)$$

Each term retains its coupling-order vector and is normalized to an explicit component basis. The supported Lorentz basis includes scalars, momenta, metrics, Dirac identity and gamma matrices, γ_5 , chiral projectors, slashed momenta, and sigma tensors. All physical axes and component orderings are explicit.

The supported colour representations are the SU(3) singlet, fundamental, antifundamental, and adjoint. Trilinear projection recognizes

$$1, \quad \delta_i^{\bar{j}}, \quad (T^a)_i^{\bar{j}}, \quad f^{abc}. \quad (38)$$

For a proposed basis tensor B_σ , projection succeeds only if

$$C_a(\mathbf{c}) = \kappa_a B_\sigma(\mathbf{c}) \quad \text{for every colour component } \mathbf{c}, \quad (39)$$

with exact scalar κ_a . Numerical resemblance is not sufficient to establish an executable colour structure.

For a trilinear term, contracting the normalized tensor with two input currents gives an oriented kernel

$$K_{r \leftarrow ij}^{A_r}(J_i, J_j) = g_{ab} \kappa_a \sum_{A_i, A_j} L_b^{A_r A_i A_j}(p_i, p_j, p_r) J_i^{A_i} J_j^{A_j}, \quad p_r = -(p_i + p_j). \quad (40)$$

Couplings, perturbative orders, colour normalization, and input/output component orderings remain attached to the kernel. The propagator is applied only when the result becomes an internal current.

13.3 Algebraic verification

Algebraic verification establishes that an optimized or factorized expression reconstructs the normalized source tensor. It does not assert that a model is unitary, renormalizable, or phenomenologically complete.

Kernel reuse, for example, requires the exact relation

$$K'(x, y; p_x, p_y) = \eta K(y, x; p_y, p_x), \quad \eta \in \{-1, +1\}, \quad (41)$$

after canonicalizing component expressions and resolving coupling aliases. When such a relation cannot be established, both kernels are retained. By contrast, a contact interaction that cannot be factorized without changing its tensor, colour coefficient, coupling, or multiplicity is rejected rather than approximated.

verification	checked identity	consequence
Colour projection	Eq. (39)	colour-flow interpretation of a kernel
Kernel equivalence	Eq. (41)	one value reused for symmetry-related contributions
Four-point factorization	exact colour topology, Lorentz components, signs, and multiplicity	two no-pole auxiliary-current stages
Fundamental Fierz relation	exact generator projection and eligible vector metadata	leading-colour singlet subtraction
Yang–Mills relations	exact cubic and quartic tensors and coupling relation	helicity and trace-reflection reuse
Tensor ordering	explicit physical axes and component permutations	stable source-to-current component maps

13.4 Four-point contact factorization

Two four-point colour structures have a verified auxiliary-current factorization. A literal colour singlet is split directly when its normalized Lorentz tensor can be reconstructed. A coloured gauge contact must have the form

$$C^{a_1 a_2 a_3 a_4} = c f^{a_i a_j b} f^{b a_k a_r}, \quad (42)$$

up to exact permutations, with one shared adjoint index b and scalar coefficient c . It is represented by a non-propagating auxiliary current,

$$A_{\rho\sigma}^b[i, j] = f^{b a_i a_j} \sum_{A_i, A_j} P_{\rho\sigma; A_i A_j} J_i^{A_i} J_j^{A_j}, \quad (43)$$

$$J_r^{A_r}|_{V_4} = \frac{c s_{ijklr}}{m_r} f^{a_r b a_k} \sum_{\rho, \sigma, A_k} F^{A_r}_{\rho\sigma A_k} A_{\rho\sigma}^b J_k^{A_k}. \quad (44)$$

Here $s_{ijklr} = \pm 1$ is the orientation parity and m_r removes indistinguishable assignments of identical inputs. The original coupling and its complete perturbative order occur only in Eq. (44); the auxiliary in Eq. (43) has unit coupling and no propagator. The factorization therefore introduces neither an artificial pole nor duplicated coupling weight.

The open-axis Lorentz tensor is reconstructed component by component. This covers the SU(3) four-vector gauge contact and supported colour-singlet four-vector contacts; other coloured four-point tensors require a separately verified intermediate colour channel.

13.5 Higher-valence colour-singlet contacts

Supported colour-singlet contacts of valence $n \geq 4$ are represented by a balanced binary tree. For input set S ,

$$A_S = \Phi_S(A_{S_L}, A_{S_R}), \quad S = S_L \dot{\cup} S_R, \quad J_r|_{V_n} = \frac{g}{m_r} \Psi_r(A_{S_L}, A_{S_R}), \quad (45)$$

where internal A_S are non-propagating, and the physical coupling g appears only in the final map. The tree has $O(n)$ auxiliary operations and $O(\log n)$ depth.

For scalar contacts, $\Phi_S(x, y) = xy$. For supported non-scalar or momentum-dependent singlet contacts, the auxiliary retains the components and momenta needed to reconstruct the original Lorentz expression, using

$$p_r = - \sum_{i \neq r} p_i. \quad (46)$$

This is generic in valence only after the colour tensor has been proved to be a singlet; it is not a general algorithm for coloured higher-point contacts.

13.6 Colour flow and model symmetries

Leading-colour singlet subtraction follows from the fundamental Fierz identity

$$(T^a)_i{}^{\bar{j}}(T^a)_k{}^{\bar{l}} = T_R \left(\delta_i{}^{\bar{l}}\delta_k{}^{\bar{j}} - \frac{1}{N_c}\delta_i{}^{\bar{j}}\delta_k{}^{\bar{l}} \right). \quad (47)$$

It is enabled only when the fundamental-generator colour tensor, fermion representations, Lorentz kernel, and massless adjoint-vector propagator satisfy the corresponding exact contracts.

Further recognizers establish vectorlike parity, Yang–Mills cubic and quartic relations, and local adjoint-current reflection. A global symmetry is used only when the reachable interaction sector is closed under the verified relations. An additional reachable interaction that does not satisfy them disables the reuse theorem but does not remove any physical contribution.

13.7 Propagators, closures, and spin two

Propagators are taken from the model definition. For standard scalar, Dirac, vector, and spin-two particles they have the schematic forms

$$P_0(p) = \frac{i}{p^2 - m^2 + im\Gamma}, \quad P_{1/2}(p) = \frac{i(\gamma^\mu p_\mu + m)}{p^2 - m^2 + im\Gamma}, \quad (48)$$

$$P_1^{\mu\nu}(p) = \frac{-i}{p^2 - m^2 + im\Gamma} \left(g^{\mu\nu} - \frac{p^\mu p^\nu}{m^2} \right), \quad P_2^{\mu\nu,\rho\sigma}(p) = \frac{i \Pi^{\mu\nu,\rho\sigma}(p)}{p^2 - m^2 + im\Gamma}. \quad (49)$$

The massless-vector branch uses Feynman gauge without the singular $p^\mu p^\nu/m^2$ term. Spin-two currents use a 16-component ordered tensor space and the massless de Donder or massive Fierz–Pauli projector. Non-propagating contact auxiliaries use the identity.

A custom UFO propagator is accepted when its numerator and denominator can be materialized in the declared component ordering. It is applied as supplied and disables optimizations that require a standard propagator. Goldstone relations are determined from quantum numbers and exact mass expressions; ambiguous matches are rejected.

Massless spin two is validated by the scalar-gravity model. Massive spin two remains experimental because its broad model and constraint coverage has not yet been qualified.

13.8 Implemented support boundary

The table distinguishes implemented classes from proof-gated or unsupported ones. “Proof-gated” means that a model is accepted when the stated exact identity can be established; it does not imply acceptance of every algebraically equivalent UFO spelling.

area	class	status	boundary
source	UFO directory / JSON	supported	UFO Python must be trusted; JSON is data-only and both use the same normalized model
spin	scalar, Dirac, vector	supported	standard wavefunctions, propagators, component orderings, and closures
spin	massless spin two	supported	16-component tensor path validated by the scalar-gravity model
spin	massive spin two	experimental	Fierz–Pauli path is implemented but broad model validation is incomplete
spin	UFO ghosts	partial	propagating ghosts require a materializable custom propagator and are not broadly qualified
spin	Majorana/FNV, spin 3/2	unsupported	no fermion-number-violating flow or Rarita–Schwinger current contract
colour	1, 3, $\bar{3}$, 8 of SU(3)	supported	leading, next-to-leading, and full-colour treatments described in this report
colour	6, $\bar{6}$, extra groups	unsupported	no sextet or multiple-group flow and contraction representation

area	class	status	boundary
arity 3	$1, \delta, T, f$ colour	supported	exact projection and materializable Lorentz components
arity 3	$d^{abc}, \epsilon, K_6, T_6$	unsupported	no executable colour-flow and contraction contract
arity 4	colour singlet	supported	exact tensor reconstruction; supported generic singlet-tree fallback
arity 4	two- f gauge colour	proof-gated	one shared adjoint index, trivalent topology, and exact Lorentz reconstruction
arity 4	other coloured tensors	unsupported	no verified auxiliary colour channel
arity > 4	colour-singlet contact	supported	balanced no-pole tree with the original coupling only at the final node
arity > 4	coloured contact	unsupported	no general irreducible-channel factorization
Lorentz	normalized tensor basis	supported	all tensor functions must normalize and materialize with explicit axes
Lorentz	form factors / unknown functions	unsupported	no portable executable function contract
propagator	standard $0, \frac{1}{2}, 1, 2$	supported	runtime mass and width, with explicit ordering and closure
propagator	custom UFO tensor	proof-gated	exact component lowering; affected symmetry reuse is disabled
symmetry	vectorlike/Yang–Mills/reflection	proof-gated	exact normalized identities over the complete reachable sector

The boundary is algebraic rather than process-name or particle-code based. Recognizing a tensor symbol during syntax inspection does not imply that its current, colour flow, or contraction is implemented. Models outside the stated boundary are rejected explicitly rather than evaluated with an inferred or approximate prescription.

References

- [1] F. A. Berends and W. T. Giele, *Recursive Calculations for Processes with n Gluons*, Nucl. Phys. B **306** (1988) 759–808, doi:10.1016/0550-3213(88)90442-7.
- [2] F. Maltoni, K. Paul, T. Stelzer, and S. Willenbrock, *Color-flow decomposition of QCD amplitudes*, Phys. Rev. D **67** (2003) 014026, doi:10.1103/PhysRevD.67.014026, arXiv:hep-ph/0209271.
- [3] C. Degrande, C. Duhr, B. Fuks, D. Grellscheid, O. Mattelaer, and T. Reiter, *UFO—The Universal FeynRules Output*, Comput. Phys. Commun. **183** (2012) 1201–1214, doi:10.1016/j.cpc.2012.01.022, arXiv:1108.2040.
- [4] L. Darmé et al., *UFO 2.0—The Universal Feynman Output format*, Eur. Phys. J. C **83** (2023) 631, doi:10.1140/epjc/s10052-023-11780-9, arXiv:2304.09883.
- [5] R. Frederix and T. Vitos, *Event generation with exponential scaling in multiplicity using AmpliCol*, JHEP **04** (2026) 192, doi:10.1007/JHEP04(2026)192, arXiv:2601.19483.